

UNIVERSIDAD DEL VALLE DE GUATEMALA



Parcial 1

Pedro Rubén Avila Cofiño, 23089
Brandon Werner Rivera Cabrera, 23088

GUATEMALA 11 de septiembre de 2026

Problema 3: Multiplicación de Matrices Densas ($C = A \times B$)

Tienen dos matrices cuadradas gigantes, A y B, de un millón por un millón de elementos. Deben calcular la matriz resultante C. Recuerden que para calcular una sola celda de C, se necesita leer toda una fila de A y toda una columna de B. ¿Cómo asignan el trabajo de llenar la matriz C de manera que minimicen la cantidad de veces que los trabajadores leen los mismos datos de las matrices originales?

- <https://github.com/Brariv/Parcial1>

Contexto y Datos

Descripción del problema elegido junto con la propuesta secuencial y la justificación de los datos de prueba utilizados (tamaño de la muestra, origen, estructuras de datos en memoria).

- El problema trata de calcular el resultado de multiplicar 2 matrices cuadradas densas A y B de tamaño $N \times N$ para obtener $C = A \times B$. Cada celda $C[i][j]$ es el producto punto de la fila i de A con la columna j de B, por lo que el algoritmo requiere N^3 multiplicaciones y sumas y N^2 escrituras en C. El costo principal no solo cubre el cómputo sino también el acceso a memoria ya que cada elemento de A y de B se necesita N veces, y la forma en que se recorren las matrices determina cuántas veces esos datos se leen una y otra vez desde memoria principal en lugar de reutilizarse desde la caché.
- Si se tratasen matrices de un millón por un millón. Una sola matriz de ese tamaño en double ocuparía aproximadamente 8 TB, por lo que no es posible cargarla en memoria de una computadora normal. Las pruebas se realizan con tamaños que sí caben en RAM y que igualmente exceden la caché del procesador, así se logra observar el comportamiento del algoritmo.

Programa Secuencial

- Se hizo la multiplicación con tres bucles anidados en el orden i luego k y luego j en lugar del orden clásico i j k. Para cada fila i se recorre k y el valor $A[i][k]$ se carga una sola vez en un registro. Con este valor se recorre j completo, acumulando $C[i][j] += A[i][k] * B[k][j]$. En este orden tanto la fila $B[k][j]$ como la fila $C[i][j]$ se leen y escriben de forma contigua en memoria. En el orden i j k la columna de B se recorre saltando de fila en fila, lo que provoca un fallo de caché por cada elemento.
- El programa mide el tiempo de la multiplicación con un timer, sin incluir la generación de datos, y reporta la suma de todos los elementos de C. Cuando N es menor o igual a 10 imprime las tres matrices para verificar manualmente el resultado.

Justificación de los datos de prueba

- Tamaño de la muestra:
 - Se usaron matrices de $N = 1000$ y $N = 2000$, con pruebas adicionales en tamaños menores para validar corrección.
 - Con $N = 2000$ cada matriz ocupa 32 MB y las tres juntas 96 MB, lo que supera la caché L2 y L3 del procesador y hace que el rendimiento dependa de la localidad de memoria, como ocurriría con las matrices $1M \times 1M$. Al mismo tiempo, el tiempo secuencial se mantiene en el orden de segundos, lo que permite repetir las corridas varias veces por integrante.
- Origen:
 - A y B se generan con `rand()` a partir de una semilla fija, con valores en el rango de 0.0 a 99.9.
 - B usa la semilla más uno para que sea distinta de A.
- Estructuras de datos en memoria:
 - cada matriz se almacena como un único arreglo unidimensional de `double` de tamaño $N * N$ en orden por filas, de modo que `M[i][j]` corresponde a `m[i * N + j]`.
 - Esto asegura que una fila completa sea contigua en memoria, evita el nivel adicional de indirección de un arreglo de punteros a filas y permite que el bucle interno se recorra de forma secuencial y vectorizable.

Estrategia de Paralelización

Luego de implementar su solución en paralelo ¿Qué directivas de OpenMP utilizaron y por qué? ¿Cómo evitaron las condiciones de carrera (race conditions) o el desbalance de carga (con el scheduling)?

- Directivas utilizadas
 - La directiva principal es `#pragma omp parallel for default(none) shared(A, B, C, n) schedule(static)` aplicada sobre el bucle externo de filas `i`.
 - `parallel for` crea el equipo de hilos y reparte entre ellos las iteraciones del bucle, en las filas de `C`.
 - Se paraleliza el bucle más externo porque es el que tiene más trabajo por iteración, lo que hace que el costo de crear y sincronizar los hilos sea despreciable comparado al cómputo de cada fila.
 - `default(none)` obliga a declarar explícitamente el ámbito de cada variable usada dentro de la región paralela.
 - `shared(A, B, C, n)` indica que las matrices y el tamaño se comparten entre hilos sin copiarse. A y B son solo lectura y C se escribe en regiones disjuntas.

- Las variables i, k, j y a_ik se declaran dentro del bucle y son privadas para cada hilo.
- schedule(static) asigna a cada hilo un bloque de filas de tamaño aproximado N dividido entre el número de hilos.
- #pragma omp simd para el bucle interno sobre j. Esta directiva le indica al compilador que no hay dependencias entre iteraciones y que puede vectorizar el bucle con instrucciones SIMD.
- Cómo se evitaron las condiciones de carrera
 - La única estructura que se escribe es C. Al repartir el trabajo por filas completas, cada hilo escribe únicamente en las filas que le fueron asignadas y ningún otro hilo toca esas posiciones.
 - A y B solo se leen, y las lecturas concurrentes no generan carreras.
 - El checksum de verificación se calcula de forma secuencial. Si se calculara con una reduction, el resultado podría variar en los últimos decimales según el orden en que se sumen los parciales en punto flotante, aunque la matriz C fuera idéntica.
- Cómo se evitó el desbalance de carga
 - Cada fila de C cuesta exactamente lo mismo: $N * N$ multiplicaciones y sumas, sin importar los valores de los datos. Como todas las iteraciones del bucle paralelizado tienen el mismo costo, un reparto estático en bloques contiguos queda balanceado por construcción.
 - No se hizo schedule(dynamic) porque la sincronización entre hilos al repartir iteraciones genera overhead que no aporta nada cuando el trabajo ya es uniforme.

Resultados y Métricas (Requisito Individual)

Esta es la sección más importante. Argumentar cómo sus decisiones realmente mejoraron su algoritmo secuencial. Para garantizar la participación equitativa, el informe debe contener las mediciones de speedup y eficiencia respectivas por cada integrante con pruebas de corridas (screenshot o video de sus ejecuciones).

- **Metodología**
 - Las mediciones se hicieron con $N = 1000$ y $N = 2000$, semilla 42, ejecutando la versión secuencial y la paralela con 1, 2, 4 y el máximo de hilos disponibles en cada máquina.
 - Cada configuración se corrió tres veces y se guardó el promedio.
 - El speedup se calcula como $S = T_{\text{secuencial}} / T_{\text{paralelo}}$ y la eficiencia como $E = S / \text{número de hilos}$.
- Mediciones Brandon Rivera
- Máquina: Macbook Air M4, 16GB Ram, MacOS, 10-core CPU

- Compilador: clang + libomp
- N = 1000

Hilos	Tiempo en segundos	Speedup	Eficiencia
1	0.139918	1.00	100%
2	0.084207	1.66	83.1%
4	0.048586	2.88	72.0%
6	0.039563	3.54	58.9%
8	0.035201	3.97	49.7%
10	0.032147	4.35	43.5%

```

> make run-paralelo N=1000 SEED=42 THREADS=1
OMP_NUM_THREADS=1 ./bin/paralelo 1000 42 1
Version:   paralelo (OpenMP)
N:         1000
Threads:   1
Tiempo (s): 0.139918
Checksum:  2495261048454.974609
> make run-paralelo N=1000 SEED=42 THREADS=2
OMP_NUM_THREADS=2 ./bin/paralelo 1000 42 2
Version:   paralelo (OpenMP)
N:         1000
Threads:   2
Tiempo (s): 0.084207
Checksum:  2495261048454.974609
> make run-paralelo N=1000 SEED=42 THREADS=4
OMP_NUM_THREADS=4 ./bin/paralelo 1000 42 4
Version:   paralelo (OpenMP)
N:         1000
Threads:   4
Tiempo (s): 0.048586
Checksum:  2495261048454.974609
> make run-paralelo N=1000 SEED=42 THREADS=6
OMP_NUM_THREADS=6 ./bin/paralelo 1000 42 6
Version:   paralelo (OpenMP)
N:         1000
Threads:   6
Tiempo (s): 0.039563
Checksum:  2495261048454.974609
> make run-paralelo N=1000 SEED=42 THREADS=8
OMP_NUM_THREADS=8 ./bin/paralelo 1000 42 8
Version:   paralelo (OpenMP)
N:         1000
Threads:   8
Tiempo (s): 0.035201
Checksum:  2495261048454.974609
> make run-paralelo N=1000 SEED=42 THREADS=10
OMP_NUM_THREADS=10 ./bin/paralelo 1000 42 10
Version:   paralelo (OpenMP)
N:         1000
Threads:   10
Tiempo (s): 0.032147
Checksum:  2495261048454.974609

```

- N = 2000

Hilos	Tiempo en segundos	Speedup	Eficiencia
1	1.043213	1.00	100.0%

2	0.634069	1.65	82.3%
4	0.421553	2.47	61.9%
6	0.339020	3.08	51.3%
8	0.308162	3.39	42.3%
10	0.292748	3.56	35.6%

```

> make run-paralelo N=2000 SEED=42 THREADS=1
OMP_NUM_THREADS=1 ./bin/paralelo 2000 42 1
Version: paralelo (OpenMP)
N: 2000
Threads: 1
Tiempo (s): 1.043213
Checksum: 19961433294837.437500
> make run-paralelo N=2000 SEED=42 THREADS=2
OMP_NUM_THREADS=2 ./bin/paralelo 2000 42 2
Version: paralelo (OpenMP)
N: 2000
Threads: 2
Tiempo (s): 0.634069
Checksum: 19961433294837.437500
> make run-paralelo N=2000 SEED=42 THREADS=4
OMP_NUM_THREADS=4 ./bin/paralelo 2000 42 4
Version: paralelo (OpenMP)
N: 2000
Threads: 4
Tiempo (s): 0.421553
Checksum: 19961433294837.437500
> make run-paralelo N=2000 SEED=42 THREADS=6
OMP_NUM_THREADS=6 ./bin/paralelo 2000 42 6
Version: paralelo (OpenMP)
N: 2000
Threads: 6
Tiempo (s): 0.339020
Checksum: 19961433294837.437500
> make run-paralelo N=2000 SEED=42 THREADS=8
OMP_NUM_THREADS=8 ./bin/paralelo 2000 42 8
Version: paralelo (OpenMP)
N: 2000
Threads: 8
Tiempo (s): 0.308162
Checksum: 19961433294837.437500
> make run-paralelo N=2000 SEED=42 THREADS=10
OMP_NUM_THREADS=10 ./bin/paralelo 2000 42 10
Version: paralelo (OpenMP)
N: 2000
Threads: 10
Tiempo (s): 0.292748
Checksum: 19961433294837.437500

```

- Mediciones Pedro Avila
- Máquina:
- Compilador:
- N = 1000

Hilos	Tiempo en segundos	Speedup	Eficiencia
1	0.141619	1.91	191.3%
2	0.075063	3.61	180.4%
4	0.040133	6.75	168.8%
6	0.031664	8.56	142.6%

8	0.032790	8.26	103.3%
12	0.032353	8.37	69.8%

- N = 2000

Hilos	Tiempo en segundos	Speedup	Eficiencia
1	4.082955	0.89	88.6%
2	1.780925	2.03	101.6%
4	0.808401	4.48	111.9%
6	0.529364	6.84	113.9%
8	0.460985	7.85	98.1%
12	0.442058	8.18	68.2%

- **Análisis de resultados**

- El escalado es consistente hasta 6 hilos (= núcleos físicos reales); de 8 a 12 hilos ya solo hay SMT compartiendo unidades de ejecución, así que la eficiencia cae a 60-70%. En N=1000 hay eficiencia >100% (superlineal) por efectos de caché al repartir bloques más pequeños entre hilos; en N=2000 con 1 hilo el paralelo es más lento que el secuencial porque el problema deja de caber en caché (96 MB) y el overhead fijo de OpenMP pesa más en ese régimen limitado por ancho de banda de memoria.